

Code Intelligence Architecture: Graphify + Postgres vs Meta Glean

How our extractor-plus-Postgres approach compares to Meta’s Glean (the LSP/SCIP lineage)

July 2026

Summary

Meta’s **Glean** is essentially “*the LSP/SCIP philosophy taken to its logical extreme*” — a compiler-grade fact database for code. Comparing our architecture (Graphify as a stateless extractor, edges and vectors in Postgres) against it clarifies exactly what we keep from that lineage and what we deliberately reject.

One-liner: Glean is “*SCIP as a compiler-grade fact database for a monorepo.*” Ours is “*SCIP’s precise core, wrapped in tree-sitter breadth, LLM cross-service recall, and vector discovery, on Postgres — because the code lives in many services, not one repo.*”

What Glean actually is

Glean is a **fact database for code**: language-specific indexers emit precise, structured facts (definitions, references, xrefs, types, inheritance, docs) into an immutable, versioned store, queried via **Angle** (a Datalog-style logic language). It powers code search, IDE navigation, and code browsing across Meta’s monorepo.

The “LSP approach” is the lineage: **LSIF -> SCIP -> Glean** are all “precompute language-server-grade results into a static index so you don’t run a live server per file.” Glean generalizes that from a symbol index into a full, queryable fact database. Crucially, its indexers lean on the **actual compiler/type-checker** (Clang for C++, Hack’s checker, etc.), so its facts are *proven*, not heuristic.

The one philosophical difference that drives everything

	Glean	Our architecture
What it stores	Only what the compiler proves	Everything, confidence-tiered (EXTRACTED / INFERRED / AMBIGUOUS)

Glean has **no notion of confidence** because it doesn't infer — it computes from the compiler. That is both beautiful and limiting: for anything the compiler cannot resolve (dynamic dispatch, reflection, **cross-service HTTP calls**), Glean simply has **no fact**, and therefore zero recall there.

Our stack deliberately fills those gaps with tree-sitter (heuristic) and an LLM (inferred) at lower confidence, because for **blast radius** a probable edge beats a missing one (the “demote-not-delete” principle). Glean would report a cross-service billing dependency as *absent*; we report it as *AMBIGUOUS*.

Full comparison

Axis	Glean (LSP / compiler)	Ours (Graphify + Postgres)
Precision source	Compiler / type-checker — exhaustive, exact	SCIP (compiler-grade, where run) + tree-sitter (syntactic) + LLM (inferred)
Storage	Bespoke immutable fact DB	Postgres (pgvector + FTS + relational edges) — commodity
Query	Angle (logic language, very expressive)	SQL + recursive CTE (analytics load in-memory)
Scope	Deep within language / repo	Coarser per-language, but spans cross-service + semantic
Cross-service (HTTP)	Not a target — monorepo resolves statically	First-class (contract-pivot inference) — Glean's blind spot
Semantic discovery	None (exact facts + code search)	Vector / FTS (PRD prose to seeds)
Confidence	N/A (only proven facts)	First-class columns — the recall backbone
Ops cost	High: compiler-integrated indexers, custom DB, Angle	Low-moderate: Postgres + tree-sitter; SCIP/LLM opt-in
Built for	Meta's monorepo, at scale	Polyglot, multi-repo microservices

Where Glean is genuinely better

- **Within-language precision and query power at scale.** With a monorepo and compiler-integrated indexers, Glean's facts are exhaustive and exact — tree-sitter plus heuristics do not come close, and Angle out-expresses SQL for complex relationship queries.
- **No “confidence” noise** — every fact is authoritative. Cleaner when you can get it.

Where ours fits the target problem and Glean does not

The target estate is **polyglot, multi-repo, cross-service** — the opposite of Meta's compiler-owned monorepo. Three things Glean structurally cannot provide there:

1. **Cross-service HTTP edges.** Glean resolves calls via the compiler; a TypeScript service calling a Python service over HTTP has *no compiler link*. That is exactly our contract-pivot

layer — and it is Glean’s blind spot, because Meta’s monorepo largely does not have this problem.

2. **Semantic discovery.** “PRD prose to which code” needs embeddings; Glean is exact-facts-only (pure LSP heritage — symbols, not semantics).
3. **Recall into the gaps.** Glean stores proven facts; it *omits* the dynamic / cross-service edges that blast radius must not miss. Our confidence tiers keep them.

Additionally, Glean is a heavy, bespoke platform; we get most of the value on **commodity Postgres** without building compiler-integrated indexers for every language.

Synthesis

We already adopt Glean’s core idea — precompute LSP/compiler-grade facts into a queryable store — **via the SCIP layer.** SCIP is the LSIF/LSP-precompute lineage, and `reconcile_scip` puts those exact facts into the graph. Where we diverge is deliberate:

Glean maximizes within-repo precision and query power on bespoke infra. We trade some of that for polyglot cross-service reach, semantic discovery, confidence-tiered recall, and commodity Postgres — because our target is not “answer exact xref queries over a monorepo,” it is “given a PRD, find every affected service, including the fuzzy edges the compiler cannot see.”

Worth borrowing from Glean: its schema rigor and incremental fact batches (immutable, versioned facts per source). Our `(repo, source)` replace-scope and typed `code_edges` are the pragmatic analog.

Not worth borrowing: Angle plus a bespoke fact DB. For a polyglot service graph with a vector layer, Postgres recursive-CTE + pgvector is the right cost/power point.